

# EEE3096S Practical 1A

Joshua Handyside  
HNDJOS012

Tebogo Teffo  
TFFTEB003

## I. TASK 1

### A. Objective

Set up TIM16 so that it interrupts the processor exactly once per second, using the chip's internal 8 MHz clock. Each interrupt must toggle pin PB0. The prescaler and ARR values had to be calculated and recorded as predictions before flashing, and the result had to be checked on an oscilloscope. A deliberate error in the supplied skeleton code also had to be found and fixed.

### B. Predictions and Calculations

| Quantity            | Predicted Value                              |
|---------------------|----------------------------------------------|
| System clock        | 8MHz                                         |
| Prescaler (PSC)     | 7999                                         |
| Auto-reload (ARR)   | 999                                          |
| Interrupt period    | 1.000s                                       |
| PB0 period          | 2.000s (0.500 Hz)                            |
| Counter clock       | $\frac{8000000}{(7999+1)} = 1000 \text{ Hz}$ |
| Interrupt frequency | $\frac{1000}{999+1} = 1 \text{ Hz}$          |

TABLE I  
TASK 1 PREDICTIONS AND CALCULATIONS

7999 was chosen over other valid combinations because it makes each tick last exactly 1 ms. The ARR value then equals the desired period in milliseconds minus one.

PB0 flips state on every interrupt, so two interrupts are needed for one complete high-low cycle. The wave on the scope was therefore expected to have double the interrupt period.

### C. Implementation

The calculated values were entered into the TIM16 setup, and PB0 was configured as a push-pull output after enabling the GPIOB clock. The timer was started with HAL\_TIM\_Base\_Start\_IT() rather than HAL\_TIM\_Base\_Start(), since only the former turns the interrupt on.

The main loop was left completely empty. This is the point of using an interrupt: the timer runs on its own in hardware and only calls the processor when a second has passed, leaving the main loop free for other work.

The deliberate error was in the supplied interrupt handler, which only instructed usto toggle the pin. When the counter resets, the timer sets a flag to signal that it needs attention. Hardware sets this flag but never clears it, so a handler that only toggles the pin returns with the flag still raised. The processor sees it again immediately and re-enters the handler,

causing PB0 to flip thousands of times per second instead of once. The fix was to clear the flag as the first line of the handler:

```
__HAL_TIM_CLEAR_IT(&htim16, TIM_IT_UPDATE);  
HAL_GPIO_TogglePin(LED_PORT, LED_PIN);
```

Clearing the flag before doing the work also means any new interrupt arriving during the handler is still recorded rather than wiped.

### D. Physical Results

A probe was placed on PB0 with the ground clip on a board ground pin. The timebase was set to 500 ms/div with normal trigger mode, and the scope's built-in frequency and period measurements were used.

| Quantity                              | Predicted  | Measured   | Error |
|---------------------------------------|------------|------------|-------|
| PB0 frequency                         | 500.00 mHz | 500.83 mHz | 0.17% |
| PB0 period                            | 2.000s     | 1.997s     | 0.15% |
| Interrupt period (half of PB0 period) | 1.000s     | 0.9985s    | 0.15% |

TABLE II  
TASK 1 RESULTS

### E. Comments and Interpretation

The measurement matched the prediction to within 0.15%, confirming the prescaler and ARR values were correct. The measured wave being twice the interrupt period also confirms the handler runs exactly once per interrupt, since a handler failing to clear its flag would produce a far higher frequency.

The signal is very slightly too fast. It cannot be a calculation error, because 7999 and 999 are whole numbers and divide exactly. The only remaining source is the clock itself. Working backwards, the actual clock is running at about 8.013 MHz (0.15% faster than assumed) rather than 8.000 MHz. This is expected: the HSI is a small oscillator built into the chip, and the datasheet allows roughly  $\pm 1\%$  drift with temperature. A 0.17% error is well inside that.

## II. TASK 2

### A. Objective

Use a timer interrupt to drive a running light across the eight LEDs on PB0 to PB7. Only one LED is lit at any moment. The lit LED moves from PB0 up to PB7, then reverses and moves back down to PB0, and repeats this bounce indefinitely. The interrupt is not allowed to touch the LEDs directly. It only sets a flag, and the main loop reads that flag and does the LED work. When the pattern turns around at each end, the edge LEDs (PB0 and PB7) must not light twice in a row.

### B. Predictions

The timer itself is unchanged from Task 1. TIM16 still interrupts once per second, so the same prescaler and ARR values were reused. The step interval (one LED to the next) will therefore be one second.

### C. Implementation

Two arrays supplied in the skeleton map each LED index to its GPIO port and pin. This lets the code refer to any LED by a single number from 0 to 7 instead of naming pins one by one.

`turn_off_all_leds()` loops through all eight LEDs and drives each pin low. This is called at the start of every update so that only one LED is ever lit.

`update_led_pattern()` does the real work. It first turns everything off (`turn_off_all_leds()`), then lights the LED at the current index, then works out which LED to light next time. A variable called `direction` holds either +1 (moving right) or -1 (moving left). While moving right, the index increases by one each step until it reaches 7, at which point the direction flips to -1. While moving left, the index decreases until it reaches 0, at which point the direction flips back to +1.

The important detail is at the turnaround. When the pattern reaches an end, the next index is set to skip past the edge rather than sitting on it. At the top, the code jumps straight to 6 instead of 7 again. At the bottom, it jumps to 1 instead of 0 again. Without this skip, the edge LED would light on two calls in a row and the pattern would stutter at each end. The skip produces the exact predicted sequence with no doubled ends. The current LED is lit first and the next index is calculated afterwards, and this ordering is what makes the skip land correctly.

The interrupt handler stays short. It clears the timer's update flag and then sets a global flag called `timer_event`:

```
__HAL_TIM_CLEAR_IT(&htim16, TIM_IT_UPDATE);  
timer_event = 1;
```

The main loop polls this flag. When it sees the flag set, it clears it back to zero and calls `update_led_pattern()`:

```
if (timer_event) {  
    timer_event = 0;  
    update_led_pattern();  
}
```

This is the non-blocking design defined in the task objective. The interrupt does almost nothing and returns quickly. All the LED work happens in the main loop, so the interrupt never gets held up and the main loop keeps full control of timing. Full code is in [1].

### D. Physical Results

The board was powered and observed by eye. One LED was lit at a time. The lit LED moved cleanly from PB0 up to PB7, reversed, and moved back down to PB0, then repeated. The observed order matched the predicted index sequence, and the edge LEDs did not light twice in a row at either turnaround. A short video was recorded as the deliverable.

### E. Comments and Interpretation

The flag-and-poll approach worked as intended. Each time the timer fired, the interrupt did only one small job: it set a flag and returned straight away. The main loop watched that flag and did the actual LED work. Splitting it this way matters because the processor is frozen inside the interrupt while it runs, so keeping the interrupt short frees the processor almost immediately for everything else. All the heavier work happens in the main loop instead. This is the standard way to handle a timed task: a quick interrupt to mark that a second has passed, and the main loop doing the real work.

The skip at each turnaround was the key piece of logic. It produced the required bounce pattern with clean ends. Removing the skip would cause the top and bottom LEDs to repeat, which is a common mistake in this kind of pattern.

## III. TASK 3

### A. Objective

Measure the mechanical bounce of the PA0 button on an oscilloscope, then write software debounce that ignores inputs during that window. A clean press toggles the running light speed between a 1s and a 0.5s step. The timer resets every time its counter reaches the ARR register, and that reset sets the step time, so the speed is changed while the program is running by writing a new value straight into ARR rather than re-running the timer setup. The prescaler stays fixed at 7999, the timer is not re-initialised, and CNT is reset to zero on each change. The pattern must change speed exactly once per press.

### B. Predictions and Calculations

Bounce was expected to settle in under 10ms, a typical figure for tactile switches. The timer clock is  $\frac{8000000}{(7999+1)} = 1000\text{Hz}$ , so one tick is 1ms and the period equals (ARR + 1)ms. Rearranging gives the required formula:

$$\text{ARR} = T_{\text{ms}} - 1$$
  
Therefore ARR for 1s step = 999  
And ARR for 0.5s step = 499

### C. Implementation

`change_timer_period()` computes `new_arr = new_period_ms - 1`, writes it to `TIM16->ARR`, and sets `TIM16->CNT = 0`. The prescaler is untouched and the timer is not re-initialised. Resetting CNT starts the next period cleanly and stops the counter ever sitting above a new, smaller ARR.

The debounce in `handle_button_press()` uses a stable for N milliseconds approach. Each loop it reads the raw pin level; any change restarts a settling window. The level must hold steady for `DEBOUNCE_MS` before it is trusted, and only on the transition into the pressed level does the code toggle `speed_state` and call `change_timer_period()` with 500 or 1000. Because the bounce burst keeps restarting the window, it is absorbed into one event, so the toggle fires once per press. The idle level of PA0 is sampled at startup, so

polarity is detected automatically and the button must not be held during power up.

#### D. Physical Results

The probe was placed on PA0 with the ground clip on a board ground pin. The scope was set to single shot trigger, the button pressed once, and the contact bounce captured on one screen (Figure X). The flat trace is PA0 at its idle level, and the cluster of fast spikes is the contact bouncing at the instant of the press before it settles. Pk-Pk read 3.13 V, matching the 3.3 V logic level. At 1.000 ms/div the burst spans a little under one division, giving a maximum bounce of roughly 1 ms.

By eye, the pattern changed speed exactly once per press with no double triggering.

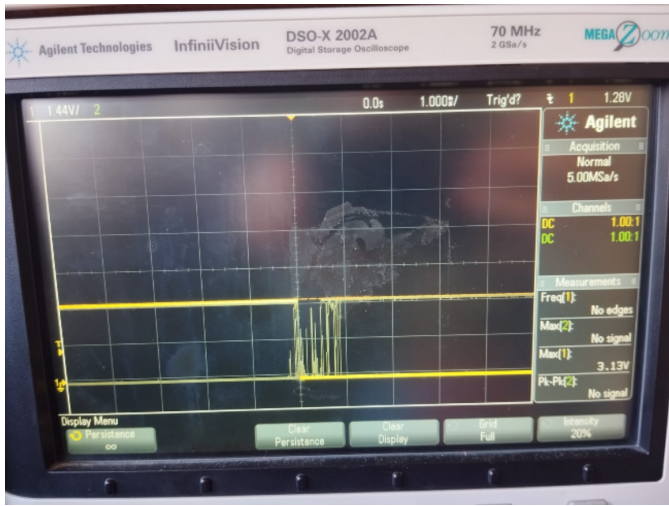


Fig. 1. PA0 contact bounce on the DSO-X 2002A at 1.000 ms/div. Idle level flat, bounce burst at the press, settling within about 1 ms

#### E. Question 4

The timer waits for the next update event. `MX_TIM16_Init` sets `TIM_AUTORELOAD_PRELOAD_ENABLE`, so `ARPE = 1` in `TIM16->CR1`. A write to `TIM16->ARR` then lands in a preload register, not the shadow register the counter compares against, so the current cycle finishes at the old period. The value transfers to the shadow register at the next update event, and the new frequency applies from the following cycle. Worst case delay is one full old period, up to 1s here.

If `ARPE` were 0 the write would hit the shadow register at once and change the period mid cycle. This is also why resetting `CNT` matters: if `CNT` had passed a new, smaller `ARR`, the compare would never match and the counter would run to `0xFFFF` before wrapping, giving one abnormally long period. To force an immediate change with `ARPE = 1`, writing `TIM16->EGR == TIM_EGR_UG` triggers a software update event, but that is not needed for a speed toggle.

#### F. Comments and Interpretation

The measured bounce of about 1ms sits well inside the expected 10ms window, so a 2ms debounce comfortably covers it with margin to spare. Waiting for the level to settle, rather than acting on the first edge, is what stopped the double triggering. Because `ARPE` is set, a press does not change the step time mid step; the current step finishes at the old rate and the new rate takes over from the next one, which is invisible on a slow pattern. Resetting `CNT` kept every period whole and avoided a glitch when switching to a shorter period.

### IV. TASK 4

#### A. Objective

Combine Tasks 1 to 3 into one program with three selectable LED modes and a shared speed control. All four buttons (PA0 to PA3) are debounced. PA0 toggles the step speed between 1s and 0.5s in every mode. PA1 selects Mode 1, the running light from Task 3. PA2 selects Mode 2, the inverse running light, where all LEDs are on except one and the dark LED moves 0 to 7 to 0. PA3 selects Mode 3, the sparkle mode, where a random pattern is shown, held for a random time, then cleared one LED at a time. The interrupt may only set a flag. All mode switching and pattern work happens in the main loop, and every mode change must clear all LEDs first. Mode 3 must be a three-state machine using `SPARKLE_IDLE`, `SPARKLE_DISPLAY`, and `SPARKLE_TURN_OFF`.

#### B. Predictions

The timer is unchanged from Task 3. `TIM16` ticks at 1000Hz (`PSC = 7999`), so `ARR = 999` gives the 1s step and `ARR = 499` gives the 0.5s step. Modes 1 and 2 advance one step per interrupt.

Mode 3 does not use the timer. Its delays are random and change every cycle, so it runs off `HAL_GetTick()` instead.

The Mode 3 LED-on time was predicted before measuring. An LED is lit at the start of a cycle, held for the full display time (100ms to 1500ms), then waits its turn to switch off, each turn-off 100ms to 150ms after the last. The shortest on-time is the minimum hold with that LED cleared first, about 100ms. The longest is the maximum hold plus waiting out every other LED, with all 8 lit:  $1500 + (7 \times 150) = 2550\text{ms}$ .

#### C. Implementation

The three modes are held in an enum (`MODE_1`, `MODE_2`, `MODE_3`, `MODE_OFF`) with a `current_mode` variable that the main loop reads to decide what to run.

The Task 3 debounce was extended from one button to four. The same stable-for-N-milliseconds logic runs in a loop over the four pins, each keeping its own settling window and confirmed level, so they debounce independently. On a confirmed press, PA0 toggles speed and PA1 to PA3 call `set_mode()`. As in Task 3, each button's idle level is sampled at startup, so no button may be held during power up.

`set_mode()` runs on every switch. It stores the new mode, calls `{ }clear_all_leds()` so nothing from the old mode is left on, and resets the tracking variables (`current_led`, `direction`, and the sparkle state) so the new mode starts clean.

Modes 1 and 2 share their movement. The 0 to 7 to 0 bounce with the turnaround skip from Task 2 was pulled into one function, `advance_index()`, used by both. `mode1_update()` clears the LEDs and lights the current one. `mode2_update()` lights all eight then turns the current one off, giving the inverse. Both then call `advance_index()`.

The ISR only clears the update flag and sets `timer_event`. The main loop reads `timer_event` for Modes 1 and 2. Mode 3 is polled on every loop pass instead, because its `HAL_GetTick` timing is finer than the timer step.

### Mode 3 state machine

Mode 3 is a switch on `sparkle_state`. A helper, `rand_range(lo, hi)`, returns an inclusive random value for both delays.

- **SPARKLE\_IDLE**: a random 8-bit value is generated with `rand()`. Each set bit lights its LED, and the lit indices are stored in an array, which is then shuffled so the LEDs clear in a random order. A random hold (100ms to 1500ms) is set as a deadline and the state moves to **SPARKLE\_DISPLAY**.
- **SPARKLE\_DISPLAY**: `HAL_GetTick()` is compared against the deadline each loop; the LEDs are untouched. Once the hold passes, the state moves to **SPARKLE\_TURN\_OFF**.
- **SPARKLE\_TURN\_OFF**: LEDs are turned off one at a time through the shuffled array, with a fresh 100 ms to 150 ms delay before each. When all are off, the state returns to **SPARKLE\_IDLE** and a new pattern starts.

The shuffle was added on purpose: without it the LEDs would always clear low to high and the effect would look mechanical. Full code is in [1].

### D. Physical Results

The board was powered and each mode checked by eye. PA1, PA2, and PA3 switched cleanly to their mode, clearing the previous pattern first. PA0 halved and restored the speed in every mode. Mode 1 ran the single-LED bounce, Mode 2 the inverse with one dark LED, and Mode 3 random patterns that held then dropped LED by LED in random order.

| Quantity            | Measured |
|---------------------|----------|
| Minimum LED-on time | 237.8ms  |
| Maximum LED-on time | 1.8s     |

TABLE III  
MEASURED MODE 3 LED-ON TIMES

A video showing all three modes and the speed toggle was recorded as the deliverable.

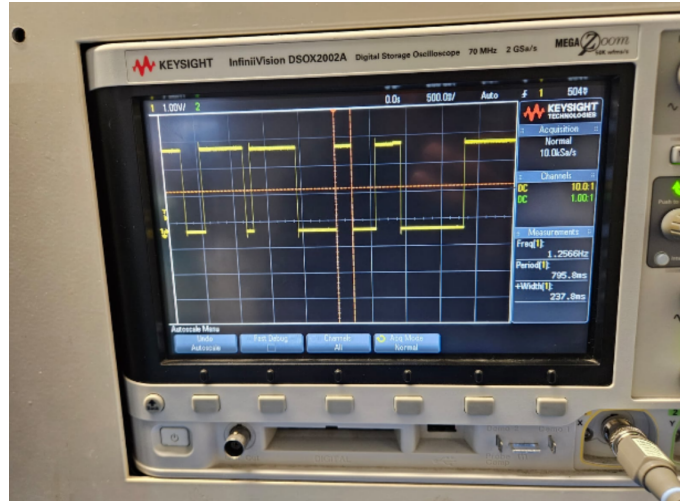


Fig. 2. Minimum LED on time

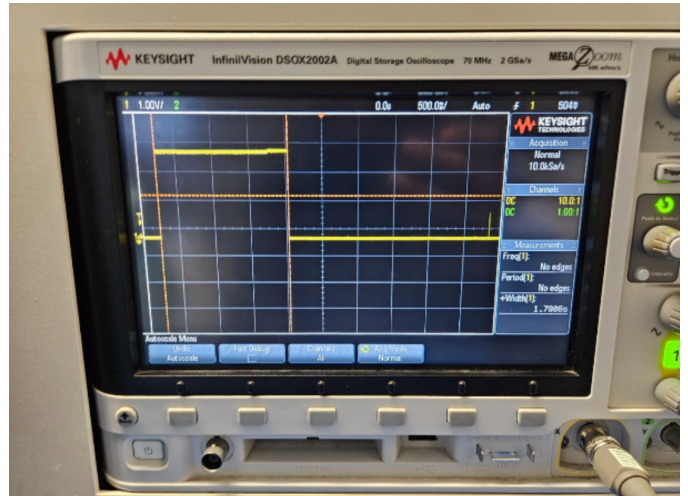


Fig. 3. Maximum LED on time

### E. Comments and Interpretation

Keeping every mode behind one `current_mode` switch and one `set_mode()` reset made the modes independent: switching never left an old LED on or a counter stuck mid-sequence. The main design choice was splitting the timing. Modes 1 and 2 are regular, so they ride on the timer flag from the ISR. Mode 3 is irregular, so it runs on `HAL_GetTick()` and is polled every loop; forcing it onto the timer would have thrown away the random delays. The measured on-times fell inside the predicted 100ms to 2550ms window, confirming the hold and turn-off ranges behave as designed.

## V. TASK 5

### A. Objective

Configure the ADC to read an external applied voltage and display a bar graph corresponding to the voltage on the 8-bit LED array.

### B. Predictions and Measurements

The predicted ADC value for an input of 2V is  $\text{round}((\frac{2.0V}{3.3V} \times 4095)) = 2482$ .

The expected input voltage during the moment the 5th LED comes on is  $\frac{2560}{4096} \times 3.3V = 2.07V$ , while the measured multimeter voltage was 2.08V.

### C. Implementation

To begin with, the GPIOB and GPIOA clocks were enabled, and pins PB0–PB7 were set to general purpose output mode to enable them to display output values. Then, the ADC was configured by enabling the clock, running calibration, selecting channel 5, and enabling the interrupt.

The display code was all within the ISR, with help from a helper function. The ISR would first read the data register and do a scaling calculation to find how many LEDs needed to be lit. Then, using the helper function, the bit pattern is written to the LEDs, displaying the voltage value as a bar.

### D. Physical Results

The results were measured by eye with the voltage at PA5 varied between 0V and 3.3V with a multimeter connected to verify. The LED array showed the entire range from 0V to 3.3V in equal steps.

### E. Comments and Interpretation

The scaling calculation initially took steps of 512 with regards to the ADC value, but this resulted in issues displaying the 8th LED. By reducing the step value slightly to 510, the voltage step values were able to be almost exactly preserved while allowing the 8th LED to display when voltage was at 3.3V.

The I/O structure for PA5 is "TTa", which means that it has a 3.3V tolerance. If a 5V DC signal were tied to PA5, then the excess voltage could destroy internal diodes or damage the ADC.

### C. Implementation

First, the GPIOB clock was enabled, and PB5 was set to output, push-pull, and high-speed mode. Then, TIM16 was initialized with PSC = 47 and ARR = 99. The interrupts for TIM16 were then configured. The IRQ handler was then set to count from 0 to 99 every interrupt. While the counter was less than 30, PB5 was driven high, and while the counter was greater than or equal to 30, PB5 was driven low.

### D. Physical Results

The results were measured using an oscilloscope to verify the frequency, period, and positive pulse width. The results from the oscilloscope were as follows:

- Frequency = 99.73Hz
- Period = 10ms
- Positive pulse width = 3ms

### E. Comments and Interpretation

The waveform produced was nearly identical to the one desired. The frequency of 99.73Hz is well within the error expected from the board's timer, which varies with temperature. Furthermore, the overshoots on spike pulses are to be expected, as the board's limited RAM will result in an imperfect square wave approximation.

If a 50Hz signal with a duty cycle resolution of 1% were required, then a period of  $1/50 = 20\text{ms}$  would be required. At a 1% resolution, this would result in an interrupt interval of  $20\text{ms}/100 = 200\mu\text{s}$ . Finally, the interrupt frequency will be  $1/200\mu\text{s} = 5\text{kHz}$ .

### REFERENCES

- [1] J. Handyside and T. Teffo, "Group-71-EEE3096S-2026," *GitHub repository*, 2026. [Online]. Available: <https://github.com/JoratorKhan/Group-71-EEE3096S-2026><https://github.com/JoratorKhan/Group-71-EEE3096S-2026>.

## VI. TASK 6

### A. Objective

Generate a PWM signal at 100Hz with a 30% duty cycle using a timer interrupt. Standard GPIO control was implemented in lieu of hardware PWM features.

### B. Predictions and Measurements

To achieve an interrupt frequency of 10kHz, the following timer values were calculated:

PSC = (8-1) = 7, given that the processor clock runs at 8MHz. This will result in the timer ticking at 1MHz.

ARR = (100-1) = 99. Given that the interrupt fires every 100μs, for a 100Hz signal, the timer must count up to 100.